

# POO en Java. Resum part 1

**MÒDUL M03**

**GRUP: DAW1 DAM1**

## Índex

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Índex</b>                                               | <b>1</b>  |
| <b>1-. Teoria: POO en Java</b>                             | <b>1</b>  |
| Sobre l'estil                                              | 1         |
| Usos de final                                              | 2         |
| Declarar constants                                         | 2         |
| Classe sense descendència                                  | 2         |
| Usos de this                                               | 2         |
| Static o no static?                                        | 2         |
| Herència i herència múltiple                               | 3         |
| Usos de super                                              | 3         |
| Public/private/protected/sense res                         | 3         |
| Sobrecarregar (overload)                                   | 3         |
| Sobreesciure (override)                                    | 4         |
| Classes abstractes                                         | 4         |
| Mètodes abstractes                                         | 5         |
| <b>2-. Exemple fet a classe: Figura, Cercle, Rectangle</b> | <b>5</b>  |
| Figura                                                     | 5         |
| Cercle                                                     | 7         |
| Rectangle                                                  | 9         |
| App                                                        | 10        |
| <b>3-. Qüestionari d'autoavaluació</b>                     | <b>11</b> |

## 1-. Teoria: POO en Java

### Sobre l'estil

Per conveni, en Java cada classe sol estar en un únic fitxer. El nom del fitxer ha de ser igual al nom de la classe. Per tant, al fitxer *Color.java* conté el codi de *class Color*, que un cop compilada genera el fitxer *Color.class*. Aquest fitxer té el bytecode, el codi que pot pot ser

interpretat per una màquina virtual Java (JVM) i que permet que una aplicació Java s'executi en qualsevol entorn.

Per evitar confusions, és important que:

1. el nom de la classe (o interfície) tingui la primera lletra en majúscules.
2. cal fer servir identificadors que facilitin la comprensió del codi. No passa res perquè siguin llargs (autocompletat).
3. les constants es posen en majúscules, separant paraules amb guions:  
`SOC_UNA_CONSTANT`
4. fem servir notació *camelCase* per a la resta. *socUnCamp*
5. cal alinear bé el codi (L'IDE ho fa automàticament si s'indica)
6. no feu servir literals, sinó constants

## Usos de **final**

Declarar constants

```
final (static) double PI= 3.1416
```

Classe sense descendència

```
final class NoPucTenirSubclasses { } //perquè sóc final
```

## Usos de **this**

1. Dins d'un mètode no estàtic, per a referir-se a un camp de l'objecte (només fa falta quan hi ha una variable amb el mateix nom). `this.nom = nom;`
2. Dins un constructor per cridar un altre constructor de la mateixa classe (ha de ser a la primera línia)

<https://docs.oracle.com/javase/tutorial/java/javaOO/thiskey.html>

## Static o no static?

Un camp/variable/constant **estàtic** o **de classe** és únic per a tota la classe. Per tant, el seu valor és compartit per a tots els objectes. Si es canvia, es canvia per a tots. Notem que dels camps/variables/constants d'instància n'hi ha una còpia independent per a cada instància de la classe.

Un **mètode és de classe o estàtic** si no depèn de cap instància concreta de la classe. Per exemple, a la classe `Math` tots els mètodes són estàtics. Actuen com funcions auxiliars o utilitats. Sembla clar que des d'un mètode estàtic no podem utilitzar el *this*, no hi ha *this*, ni els camps no estàtics de la classe (que són de cadascun dels objectes),

Els camps i mètodes de classe s'accedeixen (si són públics) posant un el nom de la classe seguit d'un punt i el nom del camp o mètode.

Per exemple, Alumnes.nAlumnes per a la següent classe:

```
class Alumnes{  
    public static int nAlumnes;  
    ....  
}
```

## Herència i herència múltiple

Quan una classe n'extén (**extends**) una altra n'hereta tots els seus mètodes i camps (no privats). Evidentment, l'herència és l'eina de reaprofitament de codi més important, ja que dissenyant una vegada els mètodes poden ser utilitzats per totes les subclasses, o fer subclasses noves. Recordem que totes les classes són subclasses de la classe **Object** ([java.lang.Object](http://java.lang.Object)), formant una jerarquia de classes on Object és el node de l'arrel.

A diferència d'altres llenguatges orientats a objecte, com per exemple el C++, el java no admet **herència múltiple** (és a dir, que una classe extengui més d'una classe). Aquest fet evita un problema de difícil de detectar: suposem que la classe C exten les classes A i B i que tant A com B tenen, cadascun d'ells, el seu propi mètode m1(). Aleshores si un objecte de la classe c crida m1, quin dels dos mètodes m1 de A o m1 de B s'ha d'executar. Si s'abusa de l'herència múltiple resulta difícil de preveure el comportament del nostre programa.

## Usos de super

De manera similar al *this*, hi ha dos usos de super.

1. Per a referir-nos a mètodes o camps visibles (public/protected) de la superclasse que s'han sobreescrit. D'aquesta manera, super.toString() executarà el toString() de la superclasse. I super.super.toString() executarà el toString() de la super classe de la superclasse, si existeix.
2. Per a cridar un constructor de la super classe (ha d'estar a la primera línia del constructor).

## Public/private/protected/sense res

<https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>

En general, es recomana que totes les propietats siguin *private* i s'accedeixin mitjançant mètodes.

## Sobrecarregar (overload)

Dos mètodes, incloent els constructors, poden tenir el mateix nom sempre que tinguin **signatures diferents** (diferent quantitat o tipus d'algun paràmetre). Notem que no es té en compte el valor de retorn. Hem vist exemples en els quals hem sobrecarregat el constructor. Per exemple, el mètode `min` està sobrecarregat 4 vegades:

```
static double    min(double a, double b)
                Returns the smaller of two double values.

static float     min(float a, float b)
                Returns the smaller of two float values.

static int       min(int a, int b)
                Returns the smaller of two int values.

static long      min(long a, long b)
                Returns the smaller of two long values.
```

## Sobreescriure (override)

Un mètode se sobreescriu quan se'n fa una nova implementació en una subclasse (no reaprofitem el mètode de la superclasse). Per exemple, el toString. Quan es sobreescriu un mètode és important fer servir l'annotació @Override. A l'exemple podem veure el mètode suma que sobreescriu un altre mètode suma (i que utilitza el suma de la superclasse).

```
@override
public int suma(int a, int b, int c){
    return super.suma(a, b) + c;
}
```

Noteu que quan sobreescrivim un mètode només afecta el mètode que té la mateixa signatura i, per tant, no afecta als mètodes sobrecarregats a la superclasse, si n'hi ha.

## Classes abstractes

```
class Figura {
    ...
}

class Cercle extends Figura {
    ...
}

class Rectangle extends Figura {
    ...
}
```

A l'exemple Figura (Figura), on Cercle i Rectangle són subclasses de Figura, ens podem preguntar si té sentit crear instàncies de Figura sense saber qui tipus de figura és. Si considerem que no té sentit, podem declarar la classe Figura com a abstracte.

```
abstract class Figura {  
    ...  
}
```

Aleshores no serà possible crear un objecte de tipus Figura.

Figura s = new Figura(); **//ERROR**

Compte que si que és possible assignar a una variable de tipus Figura un cercle o un rectangle.

Figura Cercle = new Cercle(); **//ES CORRECTE** ja que un Cercle és un Figura.

Figura rectangle = new Rectangle(); **//ES CORRECTE**

Ho discutirem més endavant quan parlem de polimorfisme.

## Mètodes abstractes

La classe Figura té un mètode que es diu *area*. Però segur que heu tingut dubtes sobre com implementar aquest mètode si no se sap de quina mena de figura es tracta. Una solució és declarar *area* a Figura com a abstracte (només cal posar la capçalera).

```
abstract public double area();
```

Declarant *area* com a abstracte, fem que totes les subclasses de Figura hagin d'implementar la seva pròpia *area*.

## 2-. Exemple fet a classe: Figura, Cercle, Rectangle

### Previs

En una primera versió vam fer Cercle i Rectangle sense Figura

Els Cercles i els Rectangles tenen moltes coses en comú: la posició, Point; i el color: Color

Fent la classe Figura, les classes Cercle i Rectangle podem reaprofitar el codi compartit.

### Solució amb herència

#### Figura

```
import java.awt.Color;
import java.awt.Point;
//importem la classes Color i Point ja fetes

public class Figura {
    //Definim l'estat: propietats o camps
    private Point point;
    private Color color;
    //Fem que sigui privat per a ocultar l'estat intern a les Apps que
    //facin servir
    //Aquest procés d'ocultar la informació
    // i fer-la accessible només amb Getters i Setter es diu encapsular

    private static int nFigures = 0; //Comptem el nombre d'instàncies
    //que es creen
    //Aquesta variable és estàtica. És de la classe i per tant,
    //accessible des de
    // totes les instàncies.

    /**
     * Aquest és el constructor principal (inicialitza totes les
     * propietats)
     * @param point
     * @param color
     */
    public Figura(Point point, Color color) {
        //Fem servir this per accedir el point ja que
        //la propietat point queda ocultada pel nom del paràmetre
        this.point = point;
        this.color = color;
        nFigures++;
    }

    //Getter i setters per encapsular point i color
    public Point getPoint() {
        return point;
    }

    public void setPoint(Point point) {
        this.point = point;
    }

    public Color getColor() {
        return color;
    }
}
```

```
public void setColor(Color color) {
    this.color = color;
}

//Aquest getter és d'una variable estàtica
//Per tant, el mètode que el crida és estàtic
public static int getNFigures(){
    return nFigures;
}

//El mètode toString s'acostuma a sobreesciure
//Per defecte, se n'hereta un de la classe Object
//Serveix al programador fer proves i vaure per pantalla
//totes les propietats
//
//Es sol generar de manera automàtica amb l'IDE
@Override
public String toString() {
    return "Figura{" +
        "point=" + point +
        ", color=" + color +
        '}';
}

//Exemple d'un mètode propi molt bàsic
public void queSoc(){
    System.out.println("soc una Figura");
}

public static void main(String[] args) {
    Figura f1 = new Figura(new Point(10, 10), Color.RED);
    Figura f2 = new Figura(new Point(10, 20), Color.GREEN);
    System.out.println(f1);
    //Equival a System.out.println(f1.toString())
    System.out.println(f2);
    //Fem una crida al mètode estàtic
    System.out.println(getNFigures());
    //També es pot cridar, des de fora, amb Figura.getNFigures()
}
}
```

## Cercle

```
import java.awt.*;
//Amb * importem totes les classes del paquet java.awt

public class Cercle extends Figura {
    //Com que hereta de Figura, ja té color i posició
    //Propietats que té un Cercle i no té una Figura
    private double radi; //Encapsulades
    private static int nCercles = 0; //Fem un comptador d'instàncies

    //El constructor de Cercle ha de cridar abans la part de la Figura
    public Cercle(Point point, Color color, double radi) {
        //Cridem al constructor de Figura (la superclasse)
        //Aquesta crida a super ha de ser a la primera línia
        //ja que no podem canviar coses de Cercle si no tenim la Figura
        super(point, color);
        this.radi = radi;
        nCercles++;
    }

    public double getRadi() {
        return radi;
    }

    public void setRadi(double radi) {
        this.radi = radi;
    }

    public static int getnCercles() {
        return nCercles;
    }

    @Override
    public String toString() {
        return "Cercle{" +
            "radi=" + radi +
            "}" +
            + super.toString();
    }

    //Sobreescrivim el mètode queSoc de Figura
    @Override
    public void queSoc() {
```



```
System.out.println("sóc un Cercle");  
  
}  
}
```

## Rectangle

```
import java.awt.*;  
  
public class Rectangle extends Figura {  
    private double base;  
    private double altura;  
  
    public Rectangle(Point point, Color color, double base, double  
altura) {  
        super(point, color);  
        this.base = base;  
        this.altura = altura;  
    }  
  
    public double getBase() {  
        return base;  
    }  
  
    public void setBase(double base) {  
        this.base = base;  
    }  
  
    public double getAltura() {  
        return altura;  
    }  
  
    public void setAltura(double altura) {  
        this.altura = altura;  
    }  
  
    public double area(){  
        return base * altura;  
    }  
  
    public double perimetre(){  
        return 2*(base + altura);  
    }  
  
    @Override  
    public String toString() {  
        return "Rectangle{" +  
            "base=" + base +  
            ", altura=" + altura +
```

```
        '}'  
        + super.toString();  
    }  
  
    @Override  
    public void queSoc(){  
        System.out.println("sóc un rectangle");  
    }  
}
```

## App

```
import java.awt.*;  
  
public class App {  
  
    public static void main(String[] args) {  
  
        Cercle c1 = new Cercle(new Point(20, 20), new Color(200, 50,  
0), 100);  
        System.out.println(c1);  
        Figura f1 = new Figura(new Point(), Color.BLUE);  
        Rectangle r1 = new Rectangle(new Point(), Color.RED, 10, 40);  
  
        //HERÈNCIA  
        //Un Cercle és una Figura i és un Objecte  
        System.out.println("Ets un cercle? " + (c1 instanceof  
Cercle)); // "c1 és un cercle"  
        System.out.println("Ets una Figura? " + (c1 instanceof  
Figura)); // "c1 és un c  
        System.out.println("Ets un Objecte? " + (c1 instanceof  
Object)); // "c1 és un c  
        System.out.println("Ets cercle? " + (f1 instanceof Cercle));  
        // "c1 és un cercle"  
        System.out.println("Ets una Figura? " + (f1 instanceof  
Figura)); // "c1 és un c  
        System.out.println("Ets un Objecte? " + (c1 instanceof  
Object)); // "c1 és un c  
  
        c1.queSoc();  
        f1.queSoc();  
        r1.queSoc();  
  
        Object c3 = new Cercle(new Point(), Color.RED, 10);  
  
        //POLIMORFISME i PERDUA D'IDENTITAT  
        Figura c2 = new Cercle(new Point(), Color.RED, 10);  
        Figura r2 = new Rectangle(new Point(), Color.RED, 10, 40);  
        //PERDUA D'IDENTITAT  
        //c2 ha perdut identitat. És un Cercle que es pensa que només
```

```
és una Figura
    //(està referenciat per la variable c2 que referencia una
Figura
    //r2 ha perduda identitat. És un Rectangle que es pensa que
només és una Figura

    //POLIMORFISME
    //A l'hora de la veritat, però, cadascú es comporta com el que
és
    //Podem veure dos Figura c2 i r2 que, tot i que han perdut
identitat,
    // es comporten com el que són. Tenen un comportament
polimòrfic
    c2.queSoc();
    r2.queSoc();

}
}
```

### 3-. Qüestionari d'autoavaluació

1. Què és un **objecte**?
2. Què és una **classe**?
3. Els objectes tenen estat i comportament. Com es representa l'estat? Com es representa el comportament?
4. Què és un constructor?
5. Hem definit la classe Pilota:
  - a. Es pot instanciar un objecte de la classe pilota si no té dissenyat cap constructor a la classe? En cas afirmatiu, mostra la sentència per a fer-ho.
  - b. Declara un constructor per a una classe Pilota que tingui un paràmetre: radi
  - c. Instancia un objecte de la classe Pilota amb el constructor anterior.
  - d. Sobrecarrega (**overload**) el constructor de la classe pilota que tingui, a més de radi, el paràmetre color.
  - e. Posa un exemple de l'ús de **this( ... )** amb els dos constructors anteriors.
  - f. Sobreescriu (**override**) el mètode toString.
  - g. Quina és la diferència entre sobrecarregar i sobreescriure?
6. Què vol dir que un atribut sigui estàtic (**static**)? Posa dos exemple en el quals sigui necessari que un atribut sigui estàtic.
7. A `System.out.println("Hola mon");`
  - a. Què és `System`?
  - b. Què és `out`?
  - c. Què és `println`?
8. Posa un exemple d'utilització de **this**. (sense que vagi seguit de parèntesis).
9. Quina diferència hi ha entre una propietat pública i privada (**public** i **private**)?

10. Que són els getters i setters? Quins avantatges tenen?
11. Seguint amb `System.out.println("Hola mon");`
  - a. `out` és estàtic o no estàtic?
  - b. Podria ser `System` un objecte? Per què?
  - c. `out` pot ser privat?
  - d. Suposant que `out` fos **final**, quines conseqüències tindria?
12. Què vol dir que un mètode sigui estàtic?
13. En un mètode estàtic, podem fer servir **this**? Per què?
14. En un mètode estatic es pot accedir a una variable de classe? Per què?
15. En un mètode estàtic, es pot accedir a una variable d'instància? Per què?
16. En un mètode d'instància, es pot accedir a una variable de classe? Per què?
17. Accedeix al mètode static, `nPilotes()`, de la classe `Pilota`. Tenim les pilotes `p1` i `p2`.
18. Declareu la classe `Figura` com a abstracta i expliqueu quin efecte té.
19. Suposant que la classe `figura` és abstracta, declareu el mètode `socAbstracte` a `figura`.  
Quin efecte té a les subclasses de `Figura`?
20. Quina finalitat té cadascuna de les maneres de fer comentaris en Java:
  - a. `/** ... */`
  - b. `/* ... */`
  - c. `//`
21. Explica breument com funciona el javadoc: Com es fan els comentaris, tres exemples d'etiquetes i com s'indiquen, com es genera el javadoc i com s'hi accedeix.
22. Dóna nom a la propietat "color contorn finestra" en *camelcase notation*.
23. Què és `MISSATGE_SECRET_CURT`?